

가천대학교 스마트시티학과 – 가상공간과 메타버스

Lecture 17 – Interfaces and Digital Twins Lab

Chris Ohk
utilForever@gmail.com

학습 목표

- Lecture 16의 인터페이스와 시각화를 기존 도시 프로그램에 적용한다.
- 함수 6개를 작성해 도시 상태를 표시하고 예제 관측값을 반영한다.
  - 코드 작성 → 단계별 검사 → 조건을 바꿔 비교 → 결과 확인

실행 준비

- 압축을 푼 스켈레톤 폴더에서 프로그램을 실행한다.
  - Windows: `py -m tkinter` → `py app.py`
  - macOS / Linux: `python3 -m tkinter` → `python3 app.py`
- `interface_rules.py`를 저장한 뒤 앱을 닫고 다시 실행한다.

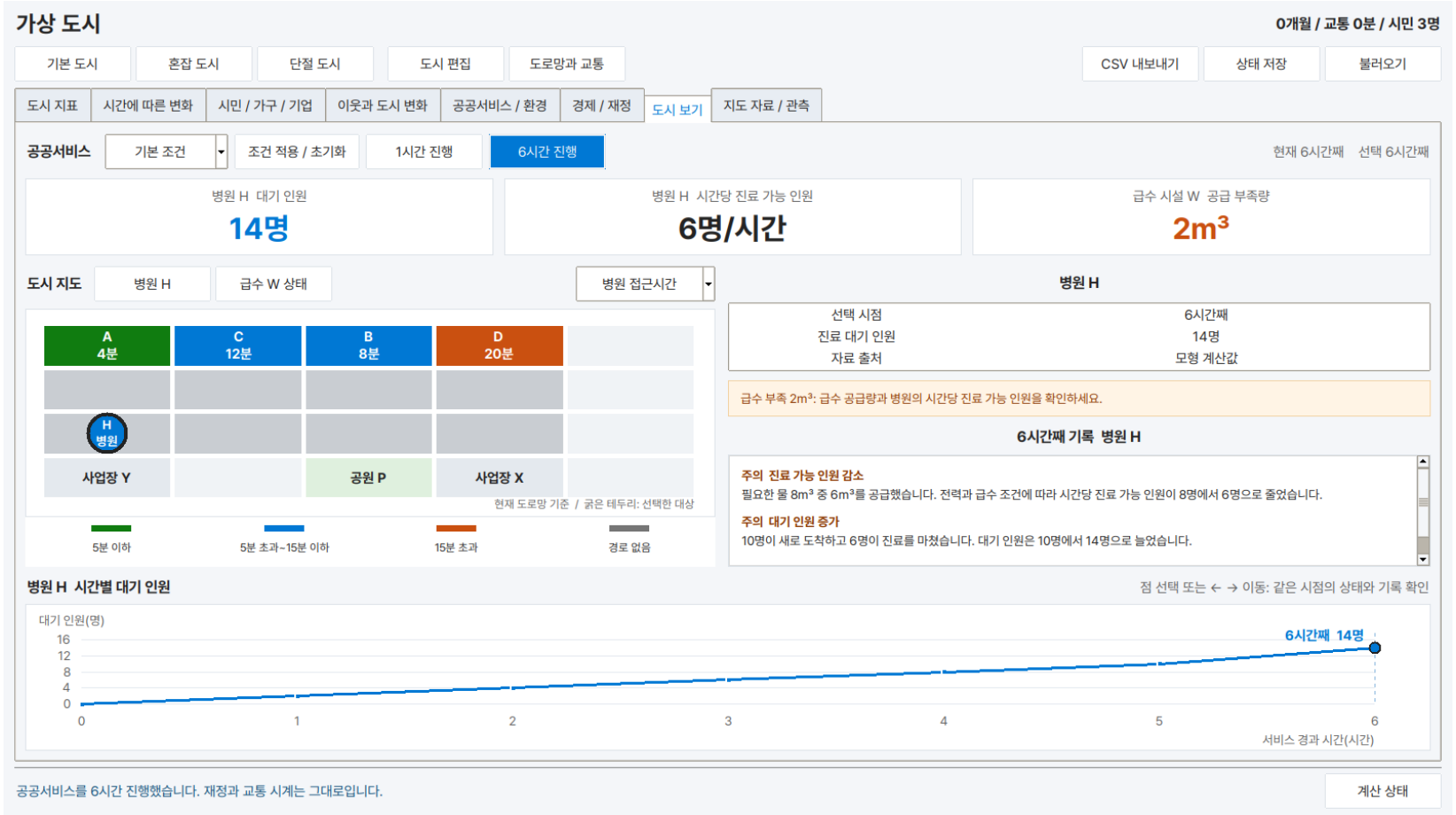
역할 나누기

- 제공된 코드에서 `interface_rules.py`의 TODO 1~6을 채운다.

파일	역할	수업 중 수정
interface_rules.py	화면에 전달할 값과 데이터 처리 규칙	직접 작성
interface_gui.py	지도 / 패널 / 그래프 / 버튼	제공
interface_support.py	기존 도시 연결 / 자료 읽기 / 저장	제공
checks_interface.py	단계별 검사 / 통합 검사	제공
기존 도시 코드	교통 / 행위자 / 서비스 / 재정	제공

완성 화면 살펴보기

- 도시 지도 / 상태 패널 / 시간에 따른 변화를 한 화면에서 확인한다.



인터페이스의 자료구조

- 화면에 표시할 정보는 딕셔너리와 리스트로 전달한다.

자료	저장하는 내용	사용할 함수
`feature`	대상 / 지표 / 값 / 단위 / 출처	`build_selection_info`
`history / events`	시간별 값 / 대상별 이벤트	`build_time_series / find_events`
지도 좌표	x, y / 원점 / 셀 크기	`point_to_cell`
`current / incoming`	현재 관측 / 새로 도착한 관측	`apply_observation`

이번 실습의 도시

- 주거지 A~D와 병원 H를 중심으로 도시 상태를 확인한다.
  - 도로로 이동하며, 병원 접근시간은 주거지마다 다르다.

A 주거지	C 주거지	B 주거지	D 주거지	물
도로	도로	도로	도로	물
H 병원	도로	도로	도로	빈 땅
Y 사업장	빈 땅	P 공원	X 사업장	빈 땅

실습 1 : 선택 정보

- `build_selection_info(feature, hour)` 함수를 작성한다.
  - 선택한 시설의 이름, 지표, 시점, 값과 단위, 자료 출처를 정리한다.
  - 반환값 : 화면에 표시할 문구를 담은 딕셔너리

선택 정보의 입력과 출력

- 병원 H의 6시간째 기록을 화면에 표시할 문구로 바꾼다.

입력 항목	입력 예	출력 예
`name / metric`	병원 H / 대기 인원	병원 H / 대기 인원
`hour`	6	6시간째
`value / unit`	14 / 명	14명
`source`	simulation	모형 계산값

0과 자료 없음의 차이

- `0`은 실제 값이고, None은 값이 없다는 뜻이다.

지표	입력값	표시할 문구
병원 대기 인원	`0`	0명
병원 대기 인원	`None`	자료 없음
병원 접근시간	`None + missing_label`	경로 없음
급수 부족량	2 / m^3	2m^3

직접 해보기 : 선택 정보

- `interface_rules.py`의 TODO 1 부분을 완성해 보자.
  - `python checks_interface.py --stage 1`
  - 예상 결과 : 14명 / 0명 / 자료 없음
- 대기 인원이 0명일 때도 값을 표시하려면 어떤 조건을 써야 할까?

선택 정보 해설

- 값이 None인지 먼저 확인한 뒤 숫자와 단위를 묶는다.

```python
def build_selection_info(feature, hour):
    value = feature["value"]

    if value is None:
        value_label = feature.get("missing_label", "자료 없음")
    else:
        value_label = f"{value:g}{feature['unit']}"

    sources = {"simulation": "모형 계산값",
               "observation": "예제 관측값"}

    return {
        "title": feature["name"],
        "metric": feature["metric"],
        "time_label": f"{hour}시간째",
        "value_label": value_label,
        "source_label": sources[feature["source"]]
    }
```

실습 2 : 지도 색상

- `layer_style(value)` 함수를 작성한다.
  - 병원까지 걸리는 시간을 구간별로 나누어 색과 범례를 정한다.
  - 갈 수 있는 경로가 없으면 **회색**과 「경로 없음」을 반환한다.
  - 반환값 : `color` / `label`

색상 구간의 경계

- `layer_style()`에 접근시간을 넣으면 색상과 범례 문구가 나온다.

입력	판정	범례 문구
0, 5	5분 이하	5분 이하
5.1, 15	5분 초과, 15분 이하	5분 초과~15분 이하
15.1, 20	15분 초과	15분 초과
None	갈 수 있는 경로가 없음	경로 없음

직접 해보기 : 지도 색상

- `interface_rules.py`의 TODO 2 부분을 완성해 보자.
  - `python checks_interface.py --stage 2`
  - 예상 결과 : 5분은 초록 / 15분은 파랑 / None은 회색
- 5분과 15분이 어느 구간에 들어가야 하는지 확인해 보자.

지도 색상 해설

- 경로가 없는 경우를 먼저 처리하고 짧은 시간부터 비교한다.

```python
def layer_style(value):
    if value is None:
        return {"color": "#767676", "label": "경로 없음"}

    if value <= 5:
        return {"color": "#107C10", "label": "5분 이하"}

    if value <= 15:
        return {
            "color": "#0078D4",
            "label": "5분 초과~15분 이하"
        }

    return {"color": "#CA5010", "label": "15분 초과"}
```

실습 3 : 시계열

- `build_time_series(history, key)` 함수를 작성한다.
  - 시간별 기록에서 시각과 선택한 지표의 값을 꺼낸다.
  - 기록의 시간 간격과 순서를 유지하고, `0`과 `None`을 구분한다.
  - 반환값 : `(hour, value)`를 담은 리스트

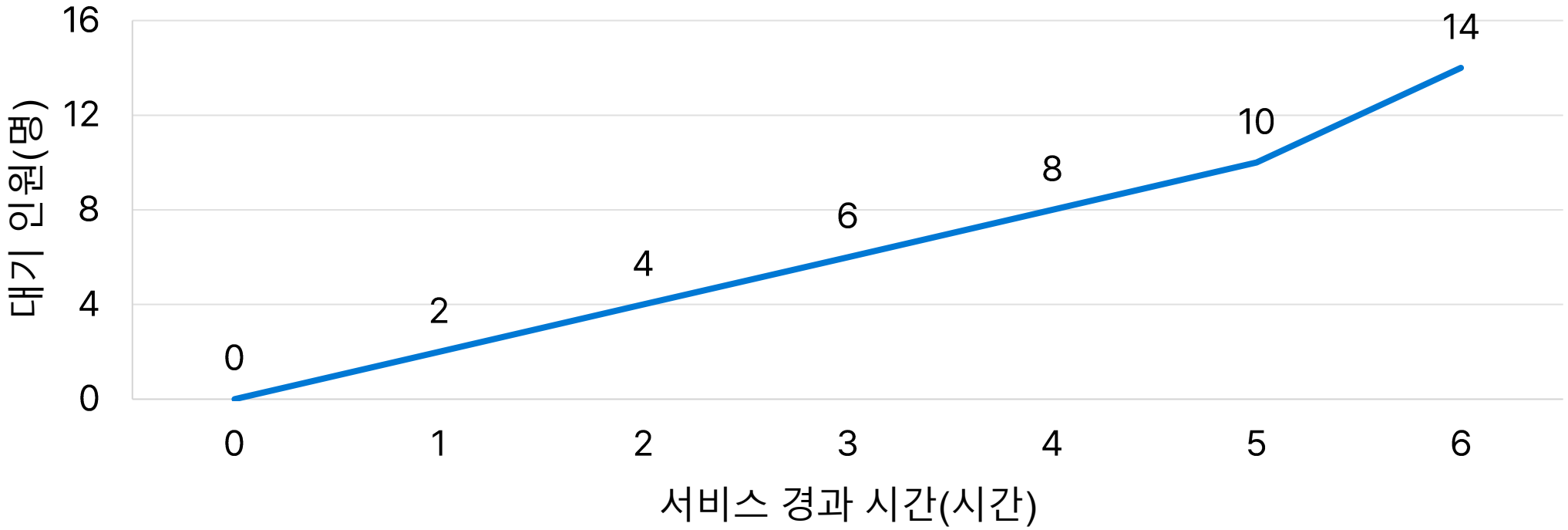
시계열 자료의 구성

- 결측값 처리용 예제에서 시간과 선택한 지표를 꺼내 보자.

입력 기록	꺼낼 값	반환되는 점
`hour=0, queue=0`	0명	`(0, 0)`
`hour=2, queue=4`	4명	`(2, 4)`
`hour=5, queue=None`	자료 없음	`(5, None)`
`hour=9, queue=10`	10명	`(9, 10)`

시간별 대기 인원

- 서비스 시간을 6시간 진행하면 병원 대기 인원이 14명이 된다.



직접 해보기 : 시계열

- `interface_rules.py`의 TODO 3 부분을 완성해 보자.
  - `python checks_interface.py --stage 3`
  - 입력 예제의 반환값 : `[(0, 0), (2, 4), (5, None), (9, 10)]`
- 자료가 없는 시간의 값을 `0`으로 바꾸면 그래프가 어떻게 달라질까?

시계열 해설

- 기록의 시간과 순서를 유지하고 값이 없으면 None을 남긴다.

```python
def build_time_series(history, key):
    points = []

    for row in history:
        points.append((row["hour"], row.get(key)))

    return points
```

실습 4 : 이벤트 검색

- `find_events(events, hour, target_id)` 함수를 작성한다.
  - 선택한 시간과 대상 ID가 모두 일치하는 기록을 찾는다.
  - 예 : 병원 H의 6시간째 점을 선택하면 H의 6시간째 기록을 찾는다.
  - 반환값 : 조건에 맞는 이벤트의 리스트

이벤트 검색의 예

- 같은 시간에 발생한 일이더라도 대상이 다르면 제외한다.

시점	대상	기록	검색 결과
5시간	H	병원 대기 증가	제외
6시간	W	물 공급 부족	제외
6시간	H	병원 처리 용량 감소	포함
6시간	H	병원 대기 증가	포함

직접 해보기 : 이벤트 검색

- `interface_rules.py`의 TODO 4 부분을 완성해 보자.
  - `python checks_interface.py --stage 4`
  - 예상 결과 : 같은 시간과 같은 대상의 기록만 반환
- `hour` 조건만 검사하면 어떤 기록이 섞여 들어올까?

이벤트 검색 해설

- 시간 조건과 대상 조건을 and로 묶는다.

```python
def find_events(events, hour, target_id):
    selected = []

    for event in events:
        if (event["hour"] == hour
                and event["target_id"] == target_id):
            selected.append(event)

    return selected
```

실습 5 : 좌표 변환

- `point_to_cell(...)` 함수를 작성한다.
  - 미터 단위 좌표를 지도의 행 번호와 열 번호로 바꾼다.
  - 반환값 : `(row, col)` / 지도 밖이면 `None`
- 제공된 지도 자료는 OSM 구조를 본뜬 가상 예제다.

좌표와 행 / 열의 차이

- 지도 좌표는 위로 갈수록 y가 커지고 화면의 행 번호는 작아진다.

구분	이번 예제의 기준
지도 크기	가로 100m × 세로 80m
셀 한 칸	20m × 20m
격자 크기	4행 × 5열
좌표의 원점	왼쪽 아래 (0, 0)
화면의 첫 셀	왼쪽 위 (0행, 0열)

한 점이 들어갈 셀

- 좌표 (45, 30)은 행 번호 2, 열 번호 2에 들어간다.
  - $\text{열} = 45 \,//\, 20 = 2, \text{행} = 4 - 1 - (30 \,//\, 20) = 2$

행 / 열	0열	1열	2열	3열	4열
0행					
1행					
2행			(45, 30)		
3행					

직접 해보기 : 좌표 변환

- `interface_rules.py`의 TODO 5 부분을 완성해 보자.
  - `python checks_interface.py --stage 5`
  - 예상 결과 : (45, 30)은 (2, 2) / 지도 밖은 `None`
- 오른쪽 경계인 $x = 100$을 허용하면 어떤 열 번호가 나올까?

좌표 변환 해설

- 지도 범위를 먼저 확인하고 y 방향을 화면의 행 번호에 맞춘다.

```python
def point_to_cell(x, y, origin_x, origin_y,
                  cell_size, rows, cols):
    local_x = x - origin_x
    local_y = y - origin_y

    if not (0 <= local_x < cols * cell_size):
        return None

    if not (0 <= local_y < rows * cell_size):
        return None

    col = int(local_x // cell_size)
    row = rows - 1 - int(local_y // cell_size)

    return (row, col)
```

실습 6 : 관측값 반영

- `apply_observation(current, incoming)` 함수를 작성한다.
  - 대상, 단위, 값의 유무, 관측 시각을 차례대로 확인한다.
  - 조건에 맞는 최신 관측값만 별도의 관측 상태에 반영한다.
  - 반환값 : `state` / `accepted` / `reason`

모형 계산값과 관측값

- 모형이 계산한 값과 관측 자료에서 읽은 값을 따로 보관한다.

구분	모형 계산값	예제 관측값
값의 근거	도시 규칙과 현재 상태	제공된 CSV의 기록
시간 기준	서비스 시작 후 경과 시간	당일 0시부터 경과한 분
이번 예제	6시간째 대기 14명	10:10 관측 대기 14명
사용 방법	도시 모형의 결과 확인	자료 반영 규칙 연습

관측 시각과 수신 시각

- 최신 여부는 자료를 관측한 시각을 기준으로 판단한다.

자료	관측 시각	수신 시각	판정
현재 값 14명	10:10	10:11	유지
새로 도착한 값 12명	10:07	10:12	과거 관측이라 제외

관측값 반영 조건

- 대상과 단위가 맞고 값이 있는 최신 관측만 반영한다.

들어온 자료	판정	이유
대상이 W	제외	target_mismatch
단위가 리터	제외	unit_mismatch
값이 None	제외	missing
관측 시각이 과거 / 동일	제외	stale / duplicate
새 관측, 값이 0	반영	Applied

직접 해보기 : 관측값 반영

- `interface_rules.py`의 TODO 6 부분을 완성해 보자.
  - `python checks_interface.py --stage 6`
  - 예상 결과 : 최신 값은 반영 / 과거, 중복, 잘못된 자료는 제외
- 원래 상태를 그대로 두고 새 상태를 만들려면 어떻게 해야 할까?

관측값 반영 해설

- 대상, 단위, 값의 유무를 순서대로 확인한다.
  그리고 현재 기록보다 나중에 관측한 값만 반영한다.

```python
def apply_observation(current, incoming):
    state = current.copy()
    reason = "applied"

    if incoming["target_id"] != current["target_id"]:
        reason = "target_mismatch"
    elif incoming["unit"] != current["unit"]:
        reason = "unit_mismatch"
    elif incoming["value"] is None:
        reason = "missing"
    elif current["observed_at"] is not None:
        if incoming["observed_at"] < current["observed_at"]:
            reason = "stale"
        elif incoming["observed_at"] == current["observed_at"]:
            reason = "duplicate"

    if reason == "applied":
        state = incoming.copy()

    return {
        "state": state,
        "accepted": reason == "applied",
        "reason": reason
    }
```

비교 실험 1 : 기본 조건

- 기본 조건을 적용하고 서비스를 6시간 진행한 뒤 병원 H를 선택한다.

확인할 항목	예상 결과	확인할 곳
병원 대기 인원	14명	상태 패널 / 그래프
병원 진료 가능 인원	6명/시간	상단 지표
급수 부족량	$2m^3$	상단 지표 / 급수 W 상태
병원 이벤트	처리 용량 감소 / 대기 증가	6시간째 기록

비교 실험 2 : 과거 시점

- 그래프에서 5시간째와 6시간째 점을 차례로 눌러 기록을 비교한다.

항목	5시간째 선택	6시간째 선택
병원 대기 인원	10명	14명
진료 가능 인원	8명/시간	6명/시간
급수 부족량	0m^3	2m^3
현재 서비스 시각	6시간째 유지	6시간째 유지

비교 실험 3 : 도로 단절

- 기본 조건과 침수 조건을 각각 3시간 진행해 접근시간을 비교한다.

주거지	기본 조건	침수 조건	지도 표시
A	4분	4분	초록
B	8분	8분	파랑
C	12분	12분	파랑
D	20분	경로 없음	주황 → 회색

비교 실험 4 : 늦은 관측

- 관측 자료를 초기화한 뒤 3번 받아 과거 기록의 처리 결과를 확인한다.

항목	현재 반영된 관측	세 번째로 받은 관측
대기 인원	14명	12명
관측 시각	10:10	10:07
수신 시각	10:11	10:12
처리 결과	14명 유지	과거 관측이라 제외

CSV 관측 기록 확인

- 나머지 관측을 받아 값의 유무와 자료의 조건을 확인한다.

CSV 순서	입력 특징	예상 처리
4번째	이미 반영한 관측 시각	중복 관측 제외
5~6번째	다른 대상 W / 다른 단위 리터	대상 / 단위가 달라 제외
7번째	값이 비어 있음	자료 없음으로 제외
8번째	최신 관측, 값이 0명	정상 반영

직접 해보기 : 도시 복원

- 기본 조건과 관측 자료를 초기화한 뒤 서비스를 6시간 진행한다.
- 관측을 3번 받고 「상태 저장」으로 JSON 파일을 저장한다.
- 서비스와 관측 자료를 초기화한 뒤 저장한 파일을 불러온다.
- 서비스 시각, 병원 기록과 예제 관측값이 복원되는지 확인한다.

완성 결과 확인

- 다음 기능이 화면과 검사 결과에서 일치하는지 확인한다.

구현한 기능	완성 후 확인할 결과
선택 정보 / 지도 색상	14명 표시 / 접근시간 구간과 범례 일치
시계열 / 이벤트 검색	선택한 시점의 값과 해당 시설의 기록 표시
좌표 변환	(45, 30) → (2행, 2열) / 지도 밖은 제외
관측값 반영	최신 관측만 반영 / 0명과 자료 없음 구분

직접 해보기 : 화면 사용성

- 다른 사람에게 화면을 보여 주고 다음 정보를 찾아보게 한다.
  - 병원 H의 6시간째 대기 인원과 그때의 기록
  - 병원에 15분 안에 도달할 수 있는 주거지
  - 현재 예제 관측값의 관측 시각과 자료 출처
- 색과 함께 표시한 숫자와 범례가 충분한지 확인한다.
- 헷갈린 항목 하나와 개선할 방법을 활동지에 적는다.

오늘 구현한 도시

- 선택한 시설의 상태를 시점, 단위, 자료 출처와 함께 표시했다.
- 지도 색상으로 접근시간을 구분하고 범례를 연결했다.
- 그래프에서 선택한 시점의 값과 이벤트 기록을 함께 확인했다.
- 지도 좌표를 격자로 바꾸고 조건에 맞는 관측값을 반영했다.

감사합니다.

utilForever@gmail.com

https://github.com/utilForever

Facebook, Twitter, Instagram, LinkedIn: utilForever